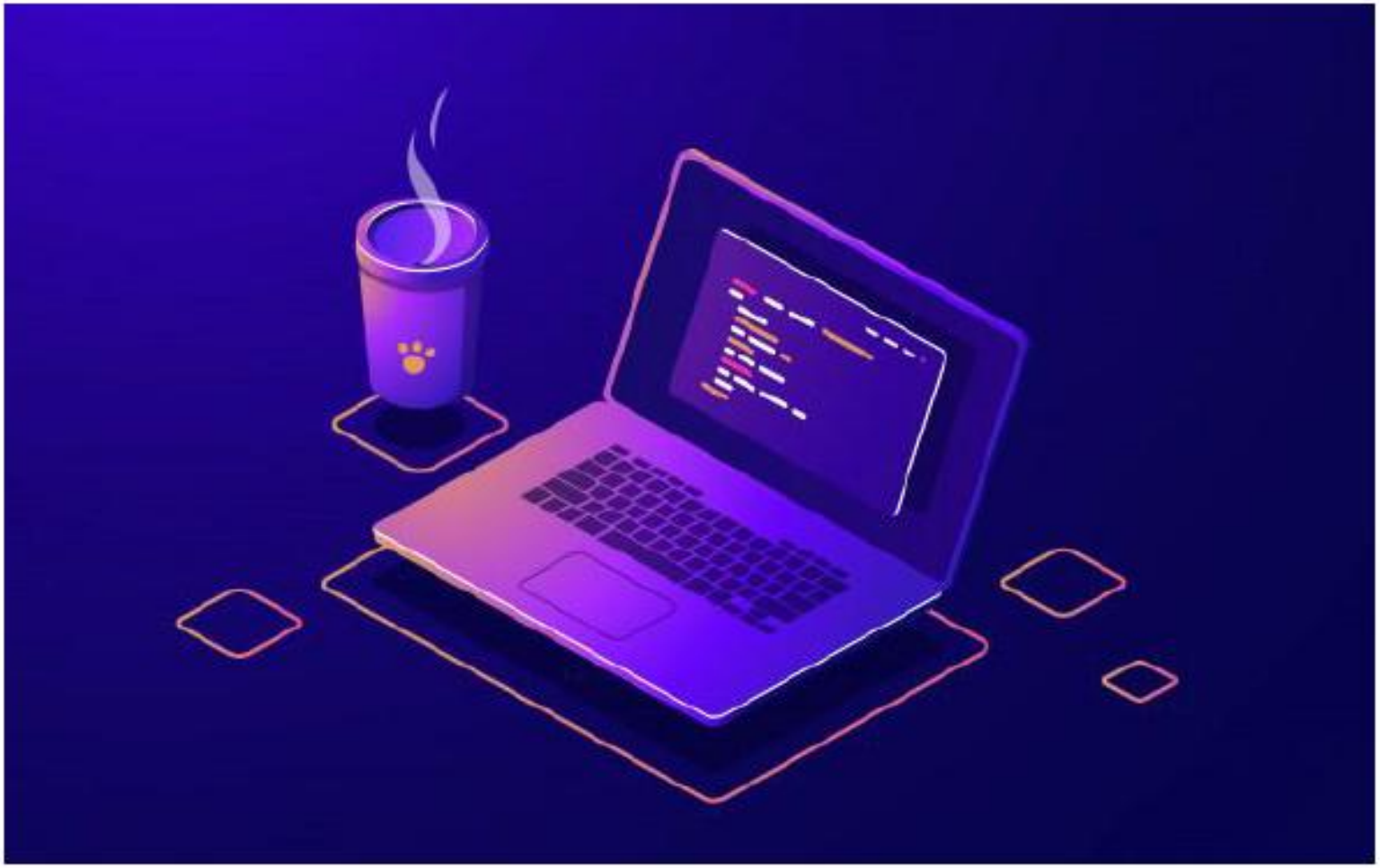


PROGRAMACIÓN

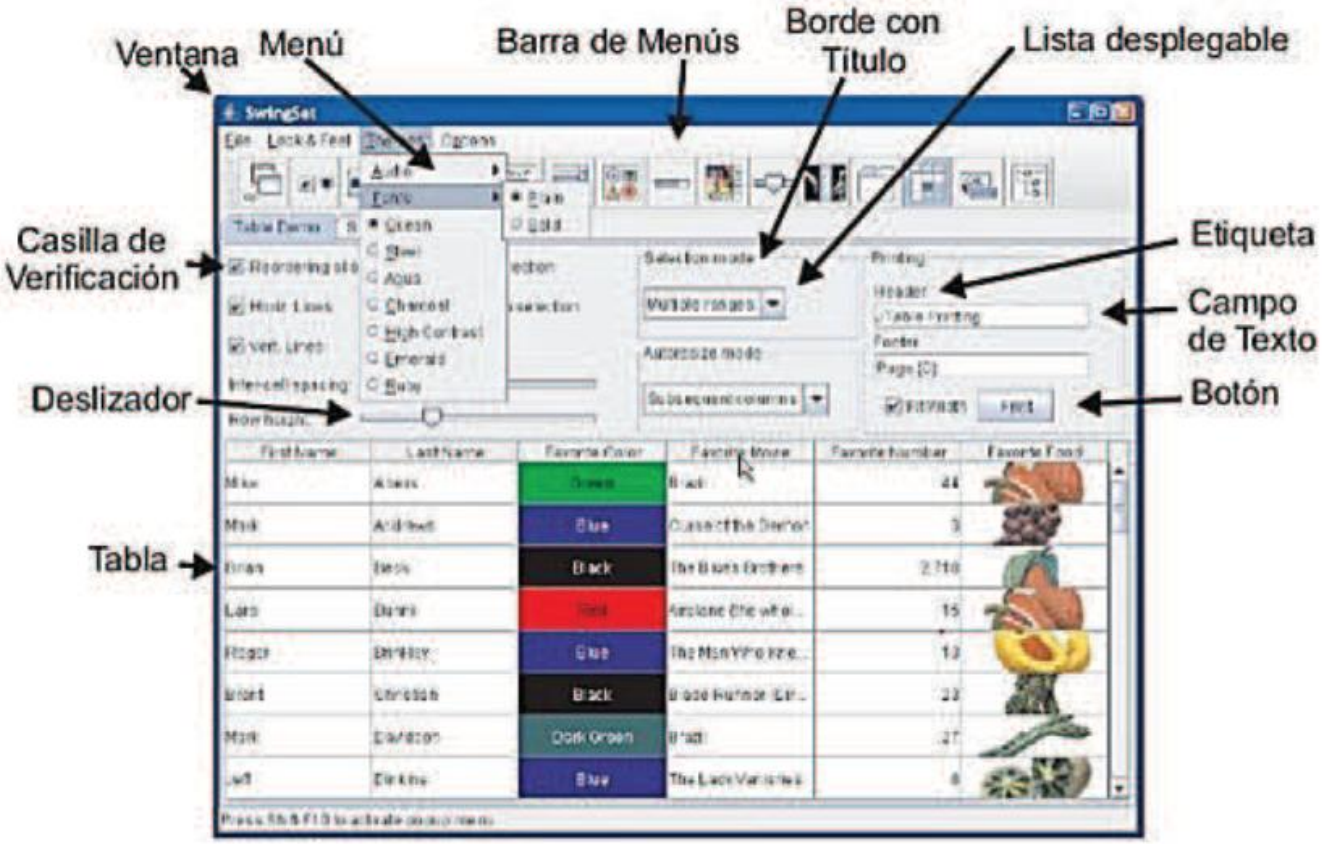
TEMA 7: INTERFACES GRÁFICAS DE USUARIO

INTERFACES GRÁFICAS DE USUARIOS

- LIBRERÍAS DE JAVA.
- APLICACIONES CON SWING.
- COMPONENTES SWING.
- CONTENEDORES SWING.
- ORGANIZACIÓN DE LOS CONTROLES EN UN CONTENEDOR.
- LAS VENTANAS Y SU APARIENCIA.
- CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS.
- GENERACIÓN DE PROGRAMAS EN ENTORNOS GRÁFICOS.

INTERFACES GRÁFICAS DE USUARIOS

Cuando nos referimos a interfaz de usuario nos referimos a una interfaz gráfica. Las interfaces gráficas se caracterizan por una serie de componentes, como botones, cajas de texto, etiquetas, paneles, barras de desplazamiento, etc. Java tiene principalmente dos librerías (APIs - Application Program Interface) bajo las cuales se pueden realizar aplicaciones con interfaz gráfica, AWT y Swing.



INTERFACES GRÁFICAS DE USUARIO

- LIBRERÍAS DE JAVA

- AWT → Abstract Window Toolkit. Parte de la JFC.

JFC → Java Foundation Classes. Clases base de Java
- Se adapta al kit de herramientas propio del entorno. Al de windows, linux.
- Se encuenrra en la librería java.AWT.
- AWT no se recomienda para nuevos desarrollos.

Se considera obsoleto para GUIs modernas, aunque algunos componentes como Graphics o Font aún se usan para gráficos básicos.

- **Swing →** Sus componentes son programados con código no nativo, + portable
- Son más potentes que los de AWT.
- Están en la librería javax.swing y son subclases Jcomponent.Swing.
- Swing sigue siendo muy usado en formación y en muchas aplicaciones de escritorio. Pero en aplicaciones nuevas, sobre todo ricas en multimedia, se prefiere JavaFX.

- **JavaFX** → Apareció en 2008 para ayudar a realizar aplicaciones ricas online. (RIA → Rich Internet Applications).
- En 2018 se convirtió en un módulo independiente de código abierto **OpenJFX**.

INTERFACES GRÁFICAS DE USUARIO

- LIBRERÍAS DE JAVA

JavaFX

Ubicación: javafx.* (desde Java 8 como módulo independiente: OpenJFX)

Ventajas:

- Soporta CSS para estilizar interfaces.
- Motor gráfico basado en hardware (más eficiente).
- Soporte de animaciones, multimedia, gráficos 2D y 3D.
- Usa FXML, un lenguaje XML para describir interfaces.
- Compatible con Scene Builder, un entorno visual para diseñar GUIs.

Estado actual: JavaFX es la alternativa moderna de Java para GUI. Aunque fue externalizado de JDK a partir de Java 11, sigue en desarrollo activo bajo el proyecto OpenJFX.

Lo más recomendado hoy día para apps nuevas es JavaFX + SceneBuilder, especialmente si quieres un diseño limpio, moderno y mantener lógica separada de la vista.

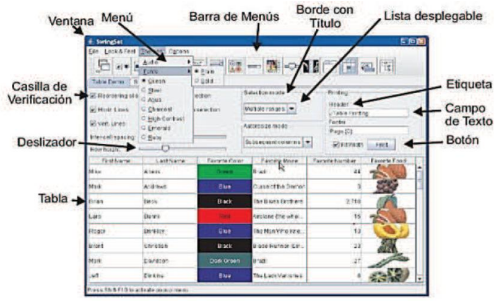
INTERFACES GRÁFICAS DE USUARIO

- LIBRERÍAS DE JAVA

Otras bibliotecas modernas a considerar:

Biblioteca	Descripción	Estado
FlatLaf	Moderno LookAndFeel para Swing que da un estilo Material Design. Muy usado.	ACTIVO
MigLayout	Layout manager muy flexible y declarativo para Swing y JavaFX.	ACTIVO
SWT (Eclipse)	Toolkit GUI alternativo, mezcla de componentes nativos y Java. Usado en Eclipse IDE.	POCO USADO
JGoodies	Conjunto de mejoras para Swing. Envejecido.	OBSOLETO
Griffon	Framework MVC para Swing, inspirado en Grails.	POCO ACTIVO

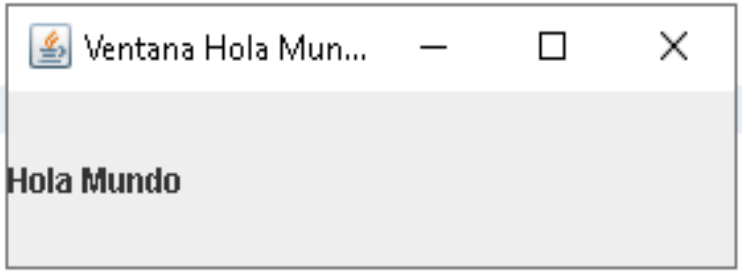
INTERFACES GRÁFICAS DE USUARIO

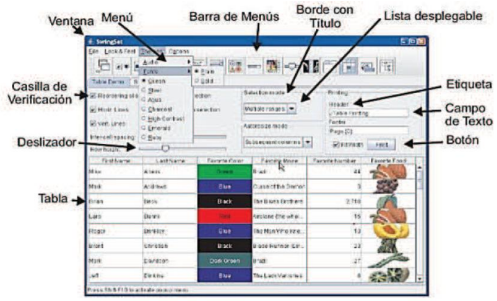


- PROGRAMA CON SWING

Realizamos un programa que muestre una ventana emergente
Con el texto “Hola Mundo”

```
import javax.swing.*;

public class holamundoswing {
    public static void main(String[] args){
        JFrame frame = new JFrame ("Ventana Hola Mundo");
        frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
        JLabel label = new JLabel("Hola Mundo");
        frame.getContentPane().add(label);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```





INTERFACES GRÁFICAS DE USUARIO

- PROGRAMA CON SWING

EXPLICACIÓN DETALLADA

Línea	Explicación
`JFrame frame = new JFrame("Hola Swing");`	Crea una ventana. El parámetro es el título que se muestra en la barra superior.
`JLabel label = new JLabel("Hola Mundo");`	Crea una etiqueta de texto.
`frame.getContentPane().add(label);`	Añade la etiqueta al "panel de contenido" del frame.
`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`	Indica que al cerrar la ventana se cierre también la aplicación.
`frame.pack();`	Ajusta el tamaño de la ventana a lo mínimo necesario.
`setLocationRelativeTo(null);`	Centra la ventana en la pantalla.
`setVisible(true);`	Hace visible la ventana.

INTERFACES GRÁFICAS DE USUARIO

- **PROGRAMA CON SWING**

  - Contenedores Swing:

    - Jframe → Implementa una ventana.

    - JDialog → Implementa una ventana de diálogos.

    - JApplet → applet → aplicación Java ejecuta en un navegador web en cliente

      JApplet es la ventana donde se ejecuta el applet.

  - frame.setDefaultCloseOperation(windowConstants.EXIT_ON_CLOSE);

    - Dice lo que hará el programa cuando el usuario cierre la pestaña.

  - Jlabel label = new Jlabel(“Hola Mundo”);

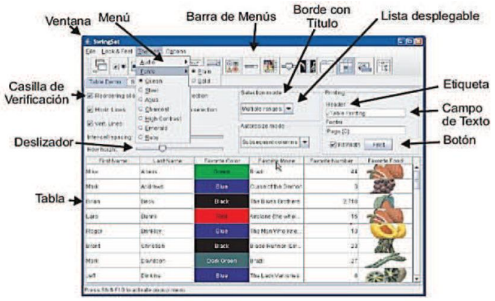
  frame.getContentPane().add(label) → añadimos elementos a la ventana.

  -frame.pack() → establece el tamaño del frame. (setSize o setBounds).

  -frame.setLocationRelativeTo(null) → Centra la aplicación en la pantalla.

  -frame.setvisible(true) → La hace visible, Otra forma → show.

INTERFACES GRÁFICAS DE USUARIO



- PROGRAMA CON SWING

MEJORA ESTÉTICA CON FLATLAF

FlatLaf es una biblioteca que moderniza la apariencia visual de Swing sin reescribir el código. Es compatible con todos los componentes clásicos.

Dependencia Maven/Gradle:

```
<!-- Maven -->
<dependency>
    <groupId>com.formdev</groupId>
    <artifactId>flatlaf</artifactId>
    <version>3.2</version>
</dependency>
```

Desde java clásico. Descargamos e instalamos el .jar (librería externa):

https://mvnrepository.com/artifact/com.formdev/flatlaf/3.5.4

License	Apache 2.0
HomePage	https://github.com/JFormDesigner/FlatLaf
Date	Dec 09, 2024
Files	pom (1 KB) jar (886 KB) View All
Repositories	Central
Ranking	**#3567 in MvnRepository (See Top Artifacts)**
Used By	**141 artifacts**

INTERFACES GRÁFICAS DE USUARIO

- PROGRAMA CON SWING

MEJORA ESTÉTICA CON FLATLAF

```
import com.formdev.flatlaf.FlatLightLaf;
import javax.swing.*;

public class HolaSwingFlatLaf {
    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(new FlatLightLaf()); // apariencia moderna
        } catch (Exception e) {
            e.printStackTrace();
        }

        JFrame frame = new JFrame("Hola Mundo con estilo moderno");
        JLabel label = new JLabel("Hola Mundo");
        frame.add(label);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

- PROGRAMA CON SWING

MEJORA ESTÉTICA CON FLATLAF

```
UIManager.setLookAndFeel(new FlatLightLaf());
```

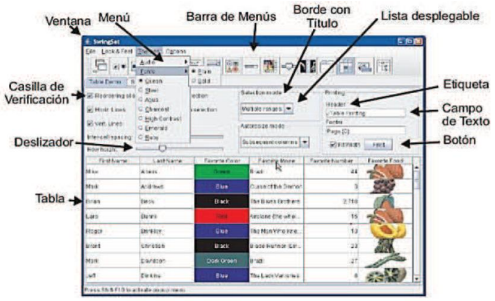
Cuando usamos esta línea, le decimos al sistema de Swing:

"A partir de ahora, todos los componentes visuales (botones, etiquetas, menús, etc.) se van a pintar y comportar como indica esta nueva apariencia llamada FlatLaf."

Técnicamente, ¿qué pasa?

1. **Swing** internamente usa una arquitectura llamada "**Pluggable Look and Feel" (PLAF):**
   - Eso significa que el diseño visual (colores, tamaños, bordes, animaciones, etc.) está desacoplado del componente en sí.
   - Un JButton no tiene una apariencia fija: depende del UIManager.
2. **FlatLaf** es una implementación moderna del **LookAndFeel**:
   - Sobrescribe cómo se pintan y comportan todos los componentes:
     - JButtonUI, JLabelUI, JTextFieldUI, etc.
   - Define fuentes, colores, bordes, animaciones, comportamiento de foco, etc.
3. Al llamar a **UIManager.setLookAndFeel**(...):
   - Swing registra una nueva estrategia de renderizado.
   - Todos los componentes que se creen después de esa línea se verán según FlatLaf.
   - Si ya había componentes creados, puedes aplicar el nuevo estilo con SwingUtilities.updateComponentTreeUI(...).

INTERFACES GRÁFICAS DE USUARIO



- PROGRAMA CON SWING

MEJORA ESTÉTICA CON FLATLAF

```
UIManager.setLookAndFeel(new FlatLightLaf());
```

Otros Temas:

```
UIManager.setLookAndFeel(new FlatDarkLaf());        // Tema oscuro
básicoUIManager.setLookAndFeel(new FlatDarculaLaf());      // Tema tipo IntelliJ
DarculaUIManager.setLookAndFeel(new FlatArcIJTheme());    // Tema elegante estilo
ArcUIManager.setLookAndFeel(new FlatGitHubDarkIJTheme());  // Tema oscuro estilo GitHub
```

Todos estos requieren que hayas añadido el .jar de los **intellij-themes**,

https://mvnrepository.com/artifact/com.formdev/flatlaf-intellij-themes/3.5.4

License	Apache 2.0
Tags	ide intellij theme
HomePage	https://github.com/JFormDesigner/FlatLaf
Date	Dec 09, 2024
Files	pom (1 KB) jar (284 KB) View All
Repositories	Central
Ranking	**#20617 in MvnRepository (See Top Artifacts)**
Used By	**21 artifacts**

INTERFACES GRÁFICAS DE USUARIO

- PROGRAMA CON SWING

EQUIVALENTE CON JavaFX + FXML

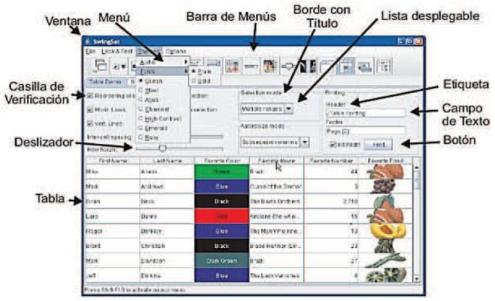
JavaFX reemplaza Swing en muchas apps modernas. Usa XML (FXML) para describir interfaces. Se puede diseñar gráficamente con Scene Builder.

Archivo ***HolaMundo.fxml***

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.StackPane?>

<StackPane xmlns="http://javafx.com/javafx"
           xmlns:fx="http://javafx.com/fxml">
    <Label text="Hola Mundo" />
</StackPane>
```

Ventaja de usar FXML	Explicación
Separación de lógica y vista	El diseño va en FXML, la lógica en Java
Permite diseño visual	Puedes usar SceneBuilder para crear GUIs arrastrando y soltando
Más limpio y escalable	Evitas mezclar diseño y código como en Swing clásico

INTERFACES GRÁFICAS DE USUARIO

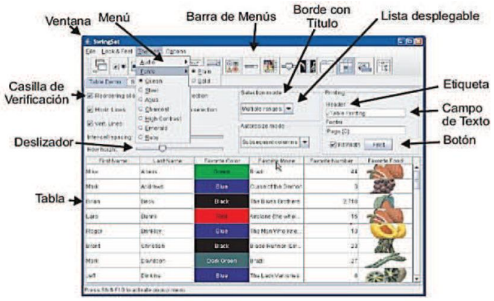


- PROGRAMA CON SWING

EQUIVALENTE CON JavaFX + FXML

```
import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class HolaJavaFX extends Application {
    @Override
    public void start(Stage primaryStage) throws Exception {
        Parent root = FXMLLoader.load(getClass().getResource("HolaMundo.fxml"));
        primaryStage.setTitle("Hola Mundo JavaFX");
        primaryStage.setScene(new Scene(root, 300, 200));
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

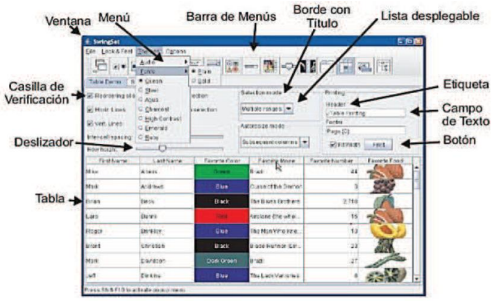


- PROGRAMA CON SWING

EQUIVALENTE CON JavaFX + FXML

- **JavaFX**: Framework de Java para crear interfaces gráficas de usuario (GUI) modernas en aplicaciones de escritorio.
- **Clase** `HolaJavaFX`: Extiende `Application` de JavaFX, lo que la convierte en una aplicación gráfica JavaFX. Define la interfaz de usuario usando un archivo FXML externo.
- **Método** `start(Stage primaryStage)`: Punto de entrada de la app JavaFX (se ejecuta al lanzarla). Aquí se construye y muestra la ventana principal:
  - `FXMLLoader.load(...)`: Carga la interfaz desde un archivo FXML (archivo XML que describe el diseño de la UI) y devuelve el nodo raíz de esa interfaz (un contenedor con los controles).
  - `Scene scene = new Scene(root)`: Crea una **escena** con el contenido cargado (usa el nodo raíz `root` como contenido de la escena).
  - `primaryStage.setScene(scene)`: Asigna la escena creada al **Stage** principal (`primaryStage`, la ventana de la aplicación).
  - `primaryStage.setTitle("Hola JavaFX")`: Establece el título de la ventana.
  - `primaryStage.show()`: Muestra la ventana en pantalla (hace visible la aplicación con su interfaz).
- **Método** `main`: Llama a `launch(args)`, método estático que inicia la aplicación JavaFX. Esto prepara el entorno gráfico y luego invoca automáticamente el método `start(...)` para arrancar la interfaz.

INTERFACES GRÁFICAS DE USUARIO



- PROGRAMA CON SWING

EQUIVALENTE CON JavaFX + FXML

Configurar JavaFX en IntelliJ (Java clásico sin Maven/Gradle)

https://gluonhq.com/products/javafx/

- Descarga el JavaFX SDK (no JMOD).
- Extrae el archivo ZIP y anota la ruta (por ejemplo: C:\javafx-sdk-21)

Añadir las librerías JavaFX al proyecto:

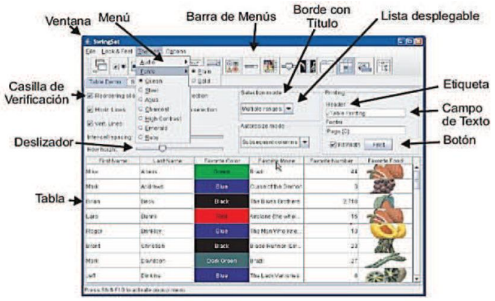
- Ve a File > Project Structure (Ctrl+Alt+Shift+S)
- Pestaña Libraries > + > Java
- Navega a la carpeta lib/ dentro del SDK de JavaFX (ej: C:\javafx-sdk-21\lib)
- Selecciona todos los .jar
- Asegúrate de que están aplicados al módulo

Añadir argumentos VM

- Ve a Run > Edit Configurations
- Selecciona tu clase principal
- En el campo VM Options, pega lo siguiente:

```
--module-path "C:\javafx-sdk-21\lib" --add-modules javafx.controls,javafx.fxml
```

INTERFACES GRÁFICAS DE USUARIO



- PROGRAMA CON SWING

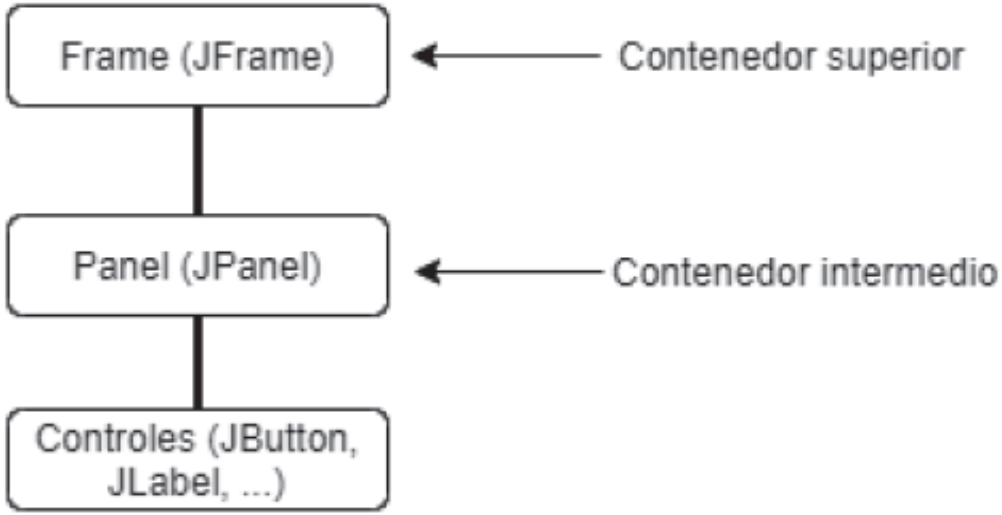
PRÁCTICAS OBSOLETAS o DEPRECADAS

Práctica	Estado	Comentario
setVisible(true) desde el main directamente sin hilos	A evitar	Debería usarse SwingUtilities.invokeLater(...) para asegurar que la GUI se maneje en el hilo de eventos.
Uso de show() en lugar de setVisible(true)	DEPRECAD0	Ya no se recomienda.
Uso de JApplet o applets en general	OBSOLETO	Eliminado de todos los navegadores modernos.
Diseño GUI 100% por código sin MVC	A evitar	No escalable. Se recomienda dividir entre modelo, vista y controlador.

INTERFACES GRÁFICAS DE USUARIO

- CONTENEDORES SWING

- Para poder incluir los componentes es preciso usar un contenedor intermedio llamado **Jpanel.**



- Los paneles son contenedores de paneles ligeros y pueden contener otros paneles.
- Pueden tener un color de fondo o cambiar la apariencia de los bordes.

INTERFACES GRÁFICAS DE USUARIO

- CONTENEDORES SWING

JFrame

Es la ventana principal de una aplicación Swing.
Tiene barra de título, botones de cerrar/minimizar, etc.
Puedes añadirle componentes mediante:

```
JFrame frame = new JFrame("Mi ventana");
frame.getContentPane().add(new JButton("Hola"));
```

Nota moderna: Desde Java 1.5 puedes usar directamente **frame.add(...)** sin getContentPane().

JDialog

Ventana secundaria, normalmente modal (bloquea otras ventanas hasta cerrarse).
Útil para mostrar mensajes, formularios, confirmaciones...

```
JDialog dialog = new JDialog(frame, "Ventana de diálogo", true);
dialog.add(new JLabel("Soy un diálogo"));
dialog.pack();
dialog.setVisible(true);
```

JApplet OBSOLETO / DEPRECAD0

Era usado para insertar interfaces Java en navegadores.
El navegador descargaba el .class o .jar y lo ejecutaba en el cliente.
Ya no se usa: los navegadores modernos han eliminado el soporte para Java Applets (por razones de seguridad).
En su lugar se usa JavaFX con WebView, o tecnologías web puras (JavaScript + HTML).

INTERFACES GRÁFICAS DE USUARIO

- CONTENEDORES SWING

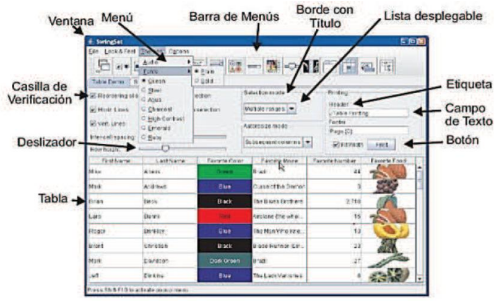
etDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);

- Indica qué debe pasar al cerrar la ventana.
- Las opciones posibles:

Constante	Acción
EXIT_ON_CLOSE	Termina la aplicación (lo más común).
DISPOSE_ON_CLOSE	Cierra la ventana, pero la app sigue funcionando si hay más abiertas.
HIDE_ON_CLOSE	Oculta la ventana sin destruirla.
DO_NOTHING_ON_CLOSE	No hace nada (útil para diálogos personalizados).

Concepto	**Explicación clara para alumnos**
JFrame	Ventana principal
JDialog	Ventana emergente (modal o no)
JApplet	Obsoleto, no se usa ya
pack()	Ajusta el tamaño automáticamente
setVisible(true)	Hace visible la ventana
setLocationRelativeTo(null)	Centra la ventana en la pantalla

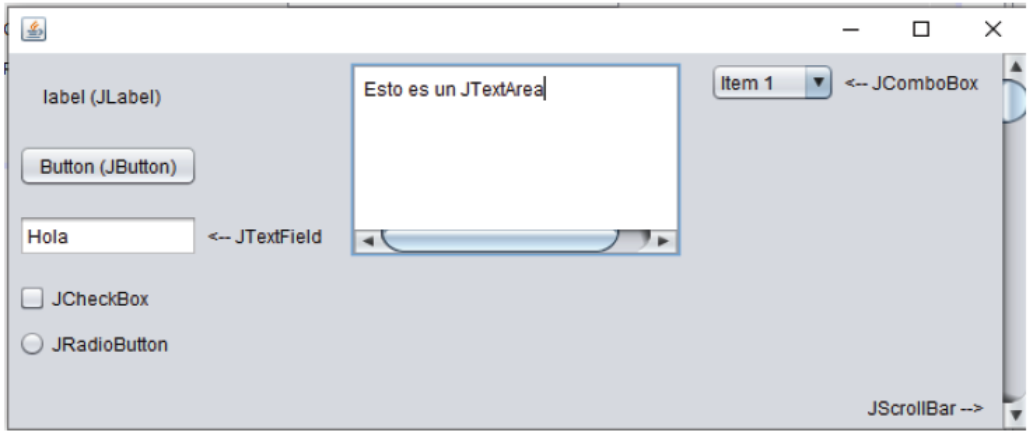
INTERFACES GRÁFICAS DE USUARIO



- **COMPONENTES SWING**

  - Son clases y por tanto se utilizan objetos en su implementación

Objeto	Descripción
JButton	Botón estándar.
JLabel	Etiqueta de texto estándar.
JTextField	Cuadro de texto.
JTextArea	Cuadro de texto multilínea.
JCheckBox	Checkbox o casilla de verificación.
JRadioButton	Radiobutton o botones de opción.
JComboBox	Lista desplegable.
JScrollBar	Barra de desplazamiento.



INTERFACES GRÁFICAS DE USUARIO

- COMPONENTES SWING

No se colocan directamente en el JFrame a pelo.
Lo correcto es crear un **JPanel** (contenedor intermedio) y añadir los componentes a él.

JPanel es un contenedor ligero para agrupar componentes.

Característica	Detalles
Es ligero	No tiene barra de título ni bordes por defecto
Es flexible	Puedes usar distintos LayoutManager
Es reutilizable	Puedes crear muchos paneles dentro de otros paneles

Ventajas del JPanel

- Puedes crear secciones lógicas de la interfaz.
- Puedes reutilizarlo en varios JFrame, JDialog, etc.
- Puedes anidar paneles: un JPanel puede contener otros JPanel.

Puedes modificar visualmente el panel:

```
JPanel panel = new JPanel();
panel.setBackground(Color.LIGHT_GRAY);      // Cambiar color de fondo
panel.setBorder(BorderFactory.createLineBorder(Color.BLACK)); // Añadir borde
```

INTERFACES GRÁFICAS DE USUARIO

- COMPONENTES SWING

Ejemplo completo:

```
import javax.swing.*;
import java.awt.*;

public class VentanaConPanel {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Ejemplo con JPanel");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel panel = new JPanel(); // contenedor intermedio
        panel.setBackground(Color.LIGHT_GRAY);

        JLabel etiqueta = new JLabel("Introduce tu nombre:");
        JTextField campo = new JTextField(15);
        JButton boton = new JButton("Enviar");

        panel.add(etiqueta);
        panel.add(campo);
        panel.add(boton);

        frame.add(panel);   // se añade el panel al frame
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

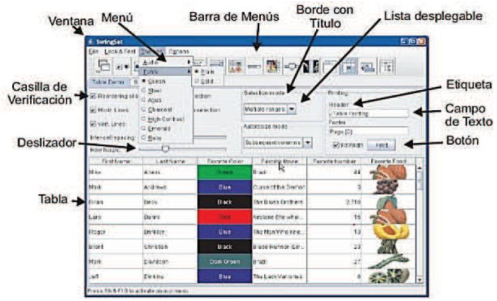
INTERFACES GRÁFICAS DE USUARIO

- COMPONENTES SWING

Ejemplo completo:

Elemento	Explicación clara
JPanel	Contenedor ligero donde van los componentes
JLabel	Texto no editable
JTextField	Caja de texto para escribir
JButton	Botón interactivo
add()	Se usa para añadir componentes al panel
setBackground()	Cambia color de fondo
BorderFactory.createLineBorder(...)	Añade bordes simples al panel

INTERFACES GRÁFICAS DE USUARIO

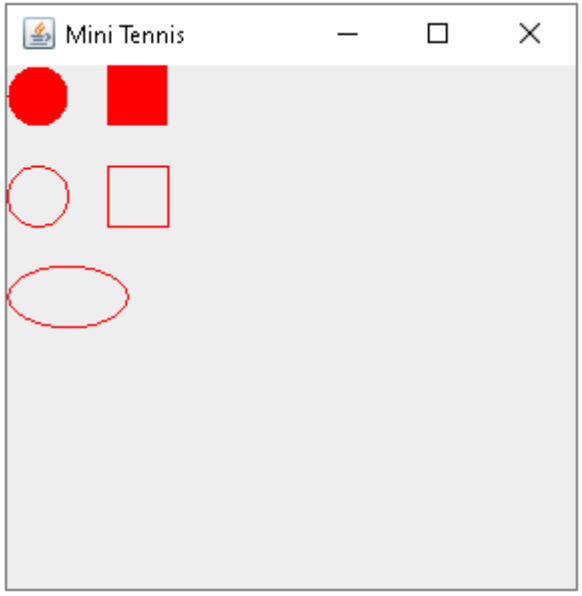


- Componentes personalizados:

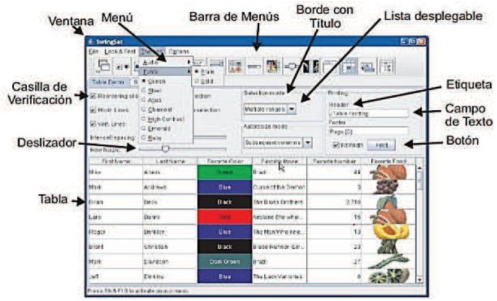
Cuando el contenido se dibuja a mano

No todos los componentes tienen que ser botones o etiquetas. A veces creamos componentes visuales que se dibujan a sí mismos, como en un juego o una simulación.

```java
import java.awt.Color;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.RenderingHints;
import java.awt.geom.Ellipse2D;
import javax.swing.JFrame;
import javax.swing.JPanel;

@SuppressWarnings("serial")
public class Game2 extends JPanel {

        @Override
        public void paint(Graphics g) {
                Graphics2D g2d = (Graphics2D) g;
                g2d.setColor(Color.RED);
                g2d.fillOval(0, 0, 30, 30);
                g2d.drawOval(0, 50, 30, 30);
                g2d.fillRect(50, 0, 30, 30);
                g2d.drawRect(50, 50, 30, 30);
                g2d.draw(new Ellipse2D.Double(0, 100, 60, 30));
        }

        public static void main(String[] args) {
                JFrame frame = new JFrame("Mini Tennis");
                frame.add(new Game2());
                frame.setSize(300, 300);
                frame.setVisible(true);
                frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```



INTERFACES GRÁFICAS DE USUARIO



- Componentes personalizados:

Cuando el contenido se dibuja a mano

Estas clases permiten:

Paquete	Uso
java.awt	Herramientas básicas de dibujo (colores, gráficos, fuentes, etc.)
java.awt.geom	Figuras geométricas 2D más precisas
javax.swing	Componentes Swing (JFrame, JPanel, etc.)

El método **paint()** es el responsable de dibujar el contenido del componente.

El objeto **g** es de tipo **Graphics**, pero lo convertimos a **Graphics2D** para tener acceso a operaciones más avanzadas (bordes, suavizado, transformaciones...).

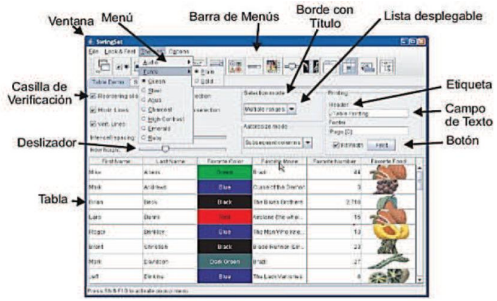
setColor define el color para todo lo que se dibuje a continuación.

Todos los métodos usan las coordenadas (x, y, ancho, alto).

Diferencia entre fill... y draw...:

Método	¿Qué hace?
fillOval(...)	Dibuja un **óvalo relleno**
drawOval(...)	Dibuja solo el **contorno** del óvalo
fillRect(...)	Rellena un rectángulo
drawRect(...)	Dibuja solo el borde
draw(new Ellipse2D.Double(...))	Usa la API de geometría 2D para dibujar una **elipse más precisa**

INTERFACES GRÁFICAS DE USUARIO



- Componentes personalizados:

Cuando el contenido se dibuja a mano

¿Por qué usar **Graphics2D**?

Permite operaciones avanzadas como:

- Antialiasing (suavizado de bordes)
- Rotaciones, escalado y transformaciones
- Control fino del trazo (grosor, patrón)
- Dibujo de curvas, polígonos complejos, imágenes, etc.

- Ejemplo con antialiasing:

```
g2d.setRenderingHint(
    RenderingHints.KEY_ANTIALIASING,
    RenderingHints.VALUE_ANTIALIAS_ON);
```

Otros componentes útiles en dibujo 2D

Clase	**¿Para qué sirve?**
Line2D, Rectangle2D, RoundRectangle2D, Ellipse2D	Formas geométricas precisas
AffineTransform	Aplicar rotaciones, escalados, traslaciones
Stroke	Grosor de línea, estilos punteados
AlphaComposite	Transparencia y mezcla
BufferedImage	Dibujar sobre imágenes o generar imágenes en tiempo real
Timer (javax.swing.Timer)	Para animaciones (movimiento de objetos)

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES.

Sin layout managers, tendrías que posicionar manualmente cada componente con coordenadas (como en otros lenguajes antiguos o videojuegos), lo que:

- Hace que la GUI no se adapte bien a otros tamaños de pantalla.
- Es difícil de mantener y escalar.
- No se ajusta bien si cambias el idioma (¡el texto se desborda!).

Los **Layout Managers** resuelven todo eso automáticamente.

En un **LayoutManager:**

- **FlowLayout**. Coloca los componentes en el contenedor de izquierda a derecha. Es el Layout Manager por defecto en los paneles.
- **BorderLayout**. Divide el contenedor en cinco partes (norte, sur, este, oeste y centro).
- **CardLayout** Permite colocar grupos de componentes diferentes en momentos diferentes de la ejecución del programa.
- **GridLayout**. Coloca los components en filas y columnas.
- **GridBagLayout**. Coloca los componentes en filas y columnas, pero un componente puede ocupar más de una columna.
- **BoxLayout**. Coloca los componentes en una fila o columna ajustándose.

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES.

Principales Layout Managers en Swing

- **FlowLayout (por defecto en JPanel)**

Coloca los componentes de izquierda a derecha
Cuando no caben, pasa a la siguiente línea
Se puede alinear a izquierda, centro o derecha

```
panel.setLayout(new FlowLayout(
                    FlowLayout.CENTER));
```

Útil para: botones o campos alineados horizontalmente
Poca flexibilidad con tamaños desiguales

- **BorderLayout (por defecto en JFrame)**

Divide el contenedor en 5 zonas:

NORTH, SOUTH, EAST, WEST, CENTER

```
panel.setLayout(new BorderLayout());
      panel.add(new JLabel("Norte"),
                  BorderLayout.NORTH);
```

Útil para: dividir la ventana en regiones
Solo un componente por zona (pero puedes anidar JPanels)

- **GridLayout**

Divide el panel en filas y columnas uniformes
Todos los componentes tienen el mismo tamaño

```
panel.setLayout(new GridLayout(2, 3));
                          // 2 filas, 3 columnas
```

Útil para: teclados, tableros
Sin control de alineación ni proporciones

- **BoxLayout**

Alinea los componentes en una fila (X_AXIS) o columna (Y_AXIS)

```
panel.setLayout(new BoxLayout(
                    panel, BoxLayout.Y_AXIS));
```

Útil para: formularios simples o menús verticales
No se adapta bien si los componentes son muy grandes

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES.

Principales Layout Managers en Swing

- **CardLayout**

  Muestra un panel a la vez, como si fueran páginas

```
CardLayout layout = new CardLayout();
JPanel panel = new JPanel(layout);
panel.add(panel1, "pantalla1");
layout.show(panel, "pantalla1");
```

Útil para: formularios por pasos, interfaces tipo “pantalla 1 / pantalla 2”

- **GridBagLayout (avanzado)**

  Como GridLayout pero permite celdas de distinto tamaño
  Cada componente tiene restricciones: posición, alineación, expansión...

```
panel.setLayout(new GridBagLayout());
GridBagConstraints c = new GridBagConstraints();
c.gridx = 0; c.gridy = 0;
panel.add(new JLabel("Usuario:"), c);
```

Muy flexible para interfaces complejas
Difícil de usar sin herramientas visuales (NetBeans GUI Builder, etc.)

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES:

¿Cuál elegir?

Layout	Cuándo usarlo
FlowLayout	Para alineación sencilla
BorderLayout	Para estructuras típicas: menú, contenido, pie
GridLayout	Para estructuras regulares
BoxLayout	Para listas verticales u horizontales
GridBagLayout	Para formularios complejos
CardLayout	Para cambiar entre pantallas

Alternativas modernas (ampliación)

MigLayout

```
panel.setLayout(new MigLayout());
panel.add(new JLabel("Usuario:"), "span 1");
panel.add(new JTextField(15), "wrap");
```

- Ultra flexible
- Sintaxis tipo HTML/CSS
- Mucho más fácil que GridBagLayout

Usado en herramientas profesionales como JFormDesigner, IntelliJ IDEA GUI Designer...
Necesita importar el .jar de MigLayout.

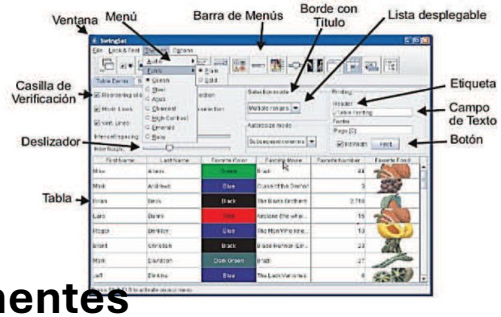
INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES: Comparación visual:

Layout	Disposición
FlowLayout	
BorderLayout	Norte / Sur / Este / Oeste / Centro
GridLayout	
BoxLayout (Y)	
CardLayout	Solo se ve una “carta” a la vez

Layout	Características clave
`FlowLayout`	Izquierda a derecha, como texto
`BorderLayout`	Norte, Sur, Este, Oeste, Centro
`GridLayout`	Tabla uniforme
`BoxLayout`	Fila o columna
`GridBagLayout`	Rejilla flexible con control total
`CardLayout`	Una “pantalla” visible a la vez
MigLayout (moderno)	Reemplazo fácil y potente para todos los anteriores

INTERFACES GRÁFICAS DE USUARIO



ORGANIZACIÓN DE LOS CONTENEDORES.

Cómo establecer Layouts y ajustar el espaciado entre componentes

Establecer el LayoutManager

Puedes asignar un layout al crear el contenedor:

```
JPanel panel = new JPanel(new FlowLayout());
```

O bien posteriormente:

```
panel.setLayout(new FlowLayout());
```

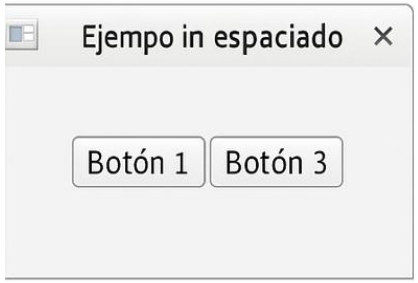
Esto permite cambiar el layout dinámicamente si es necesario.

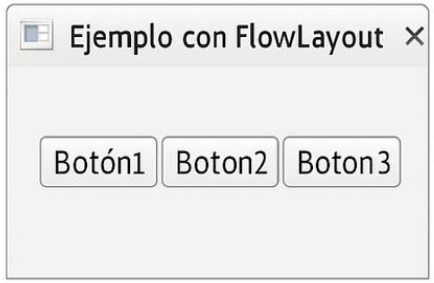
Añadir componentes con posicionamiento

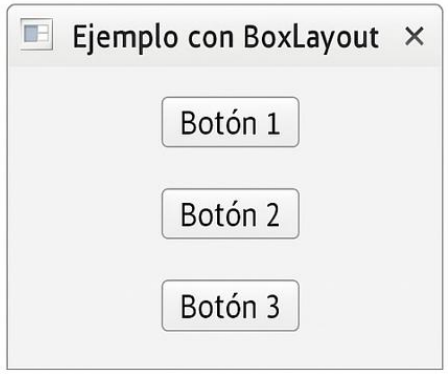
Algunos layouts (como BorderLayout) requieren especificar la zona donde se añadirá el componente:

```
panel.add(componente, BorderLayout.PAGE_START); // Equivale a BorderLayout.NORTH
```

Otros layouts, como FlowLayout o GridLayout, ignoran esa segunda posición porque colocan automáticamente según reglas internas.







INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES.

Cómo establecer Layouts y ajustar el espaciado entre componentes

Espaciado entre componentes

Cuando los componentes están demasiado juntos o demasiado separados, puedes intervenir de varias formas:

- **A) Usar parámetros del Layout**

```
panel.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 20));
```

10 → separación horizontal entre componentes
20 → separación vertical

GridLayout también permite:

```
new GridLayout(2, 3, 10, 10); // filas, columnas, hgap, vgap
```

- **B) Añadir componentes invisibles (espaciadores)**

En **BoxLayout** puedes usar:

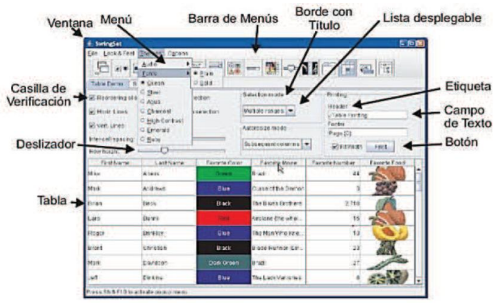
```
panel.add(Box.createRigidArea(new Dimension(20, 0))); // espacio
                                                        horizontal fijo
panel.add(Box.createVerticalStrut(15)); // espacio vertical
```

- **C) Usar bordes invisibles (EmptyBorder)**

```
componente.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
```

Esto añade margen interno: arriba, izquierda, abajo, derecha (en píxeles).

INTERFACES GRÁFICAS DE USUARIO



ORGANIZACIÓN DE LOS CONTENEDORES.

Cómo establecer Layouts y ajustar el espaciado entre componentes

Otros tipos de borde (por si se desea decorativo)

Tipo de borde	Método de creación
Línea simple	createLineBorder(Color.RED, 2)
Borde con título	createTitledBorder("Datos")
Borde compuesto (doble)	createCompoundBorder(...)
Borde vacío (espaciador)	createEmptyBorder(...)

Buenas prácticas modernas

Recomendación	Justificación
Usar pack() en lugar de setSize()	Ajusta automáticamente al contenido
Usar BoxLayout para formularios simples	Muy intuitivo y fácil de personalizar
Combinar layouts anidados	Divide la GUI en paneles con layouts distintos

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES.

Orientación de los componentes (izquierda ↔ derecha)

¿Qué es la orientación de componentes?

La orientación de los componentes en un contenedor define en qué dirección se colocan los elementos:

- Por defecto, de izquierda a derecha (LEFT_TO_RIGHT)
- Pero puede cambiarse a derecha a izquierda (RIGHT_TO_LEFT)

Esto es útil para:

- Idiomas que se escriben de derecha a izquierda (árabe, hebreo)
- Efectos visuales particulares
- Imitar diseño móvil (botón de atrás a la derecha)

¿Cómo se cambia la orientación?

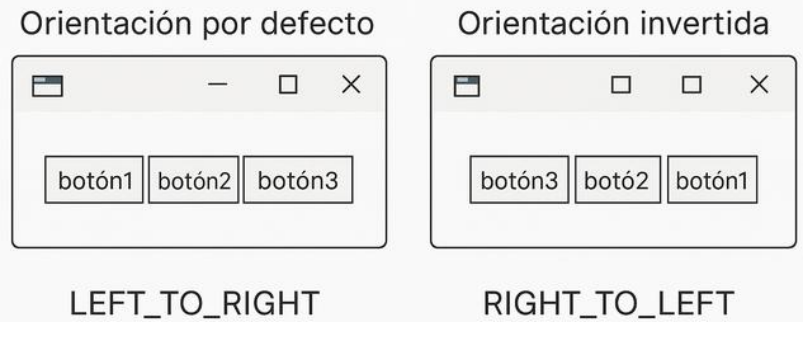
Puedes usar:

```
contenedor.applyComponentOrientation(ComponentOrientation.RIGHT_TO_LEFT);
```

O volver al modo por defecto:

```
contenedor.applyComponentOrientation(ComponentOrientation.LEFT_TO_RIGHT);
```

Se aplica tanto a JFrame como a JPanel.



INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class OrientacionDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Ejemplo de orientación");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel panel = new JPanel(new FlowLayout());
        panel.add(new JButton("boton1"));
        panel.add(new JButton("boton2"));
        panel.add(new JButton("boton3"));

        // Cambiar a orientación de derecha a izquierda:
        panel.applyComponentOrientation(ComponentOrientation.RIGHT_TO_LEFT);

        frame.add(panel);
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

- ORGANIZACIÓN DE LOS CONTENEDORES.

Apariencia de las ventanas (Look and Feed)

Uso de UIManager para cambiar el estilo visual general de los componentes Swing

¿Qué es el Look and Feel?

En Java Swing, el Look and Feel (apariencia y comportamiento) define cómo se ven y se comportan visualmente los componentes: colores, bordes, fuentes, iconos, animaciones...

Java proporciona varios estilos por defecto, pero también permite usar librerías externas para obtener interfaces modernas y profesionales.

Cambiar el estilo con UIManager

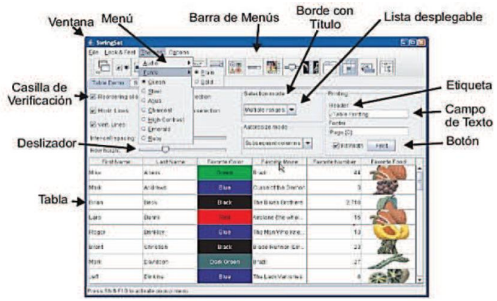
Puedes usar UIManager.setLookAndFeel(...) para aplicar un estilo global a tu aplicación.

Ejemplo básico:

```
UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");
```

O dinámicamente:

```
UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
```



INTERFACES GRÁFICAS DE USUARIO

- ORGANIZACIÓN DE LOS CONTENEDORES.

Apariencia de las ventanas (Look and Feed)

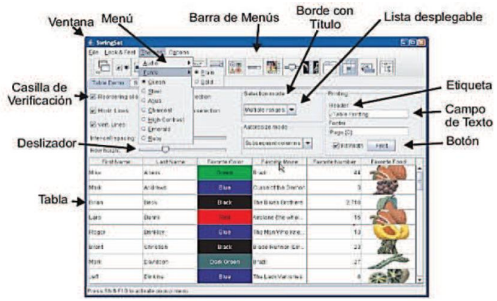
Estilos disponibles en Java estándar

Estilo	Clase Java	Comentarios
Metal	javax.swing.plaf.metal.MetalLookAndFeel	Estilo por defecto (anticuado)
Nimbus	javax.swing.plaf.nimbus.NimbusLookAndFeel	Más moderno, recomendado si no se usa FlatLaf
Motif	com.sun.java.swing.plaf.motif.MotifLookAndFeel	Muy antiguo (X11, Unix)
Windows (en Windows)	com.sun.java.swing.plaf.windows.WindowsLookAndFeel	Estilo nativo de Windows
GTK+ (en Linux)	com.sun.java.swing.plaf.gtk.GTKLookAndFeel	Nativo de sistemas GTK

¿Cómo saber qué estilos tienes disponibles?

Puedes usar este código:

```
for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
    System.out.println(info.getName() + " -> " + info.getClassName());
}
```

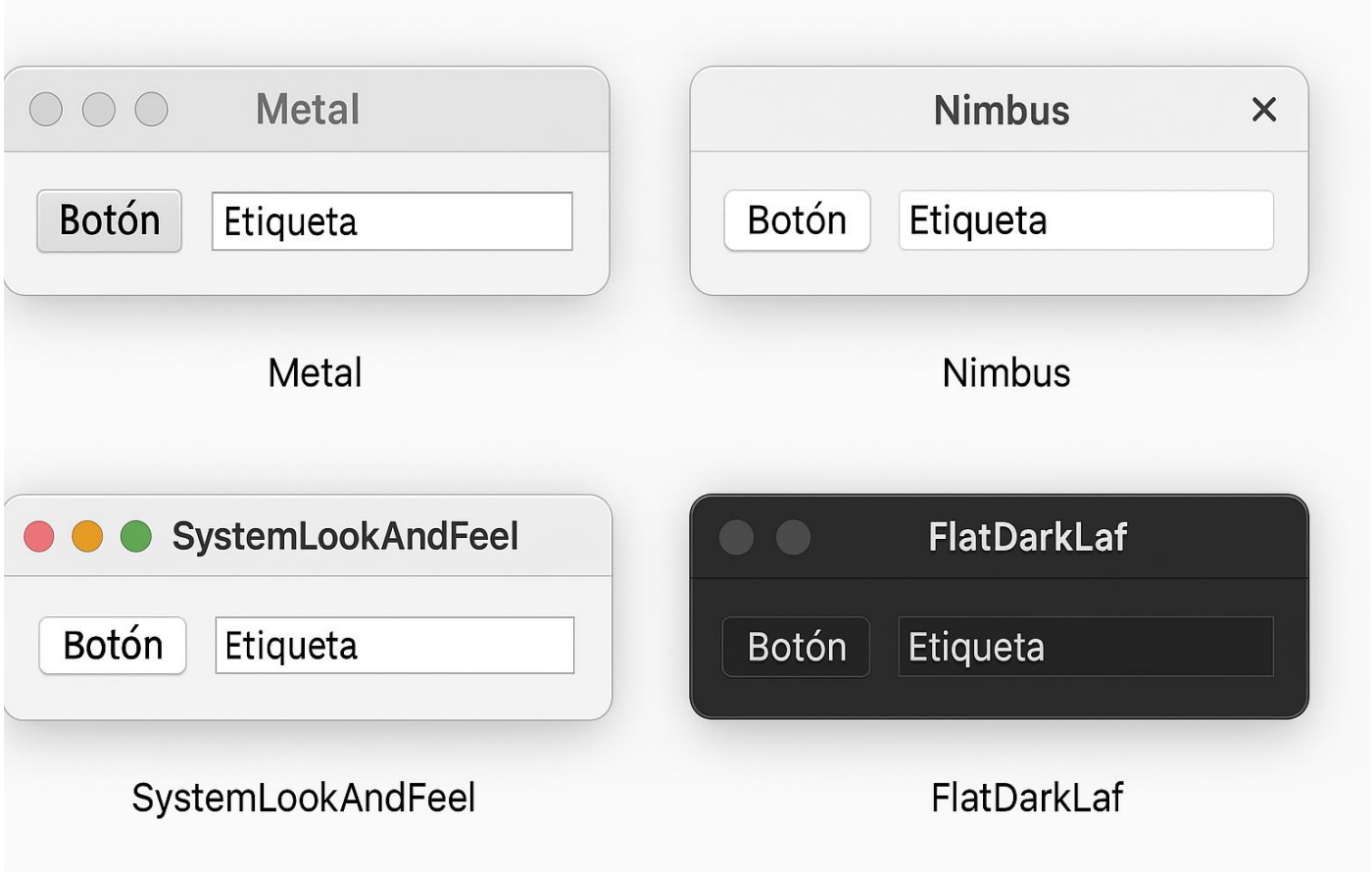


INTERFACES GRÁFICAS DE USUARIO

- ORGANIZACIÓN DE LOS CONTENEDORES.

Apariencia de las ventanas (Look and Feed)

Estilos disponibles en Java estándar



INTERFACES GRÁFICAS DE USUARIO

- ORGANIZACIÓN DE LOS CONTENEDORES.

Apariencia de las ventanas (Look and Feed)

Estilos modernos recomendados: FlatLaf

FlatLaf es un Look and Feel moderno, ligero y profesional.

Tema disponible en FlatLaf

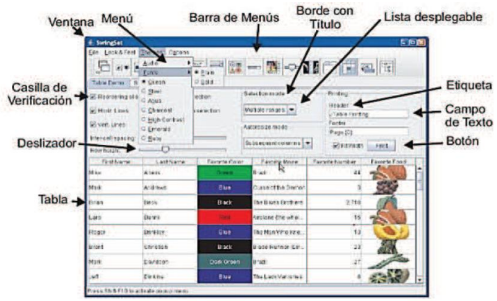
```
FlatLightLaf
FlatDarkLaf
FlatIntelliJLaf
FlatDarculaLaf
FlatArcDarkIJTheme (requiere .intellij-themes)
```

Ejemplo con FlatLaf:

```
UIManager.setLookAndFeel(new FlatDarkLaf());
```

Buenas prácticas actuales

Recomendación	Motivo
Usar FlatLaf o Nimbus	Metal está obsoleto visualmente
Aplicar el Look and Feel **antes** de crear los componentes	Para que se rendericen correctamente

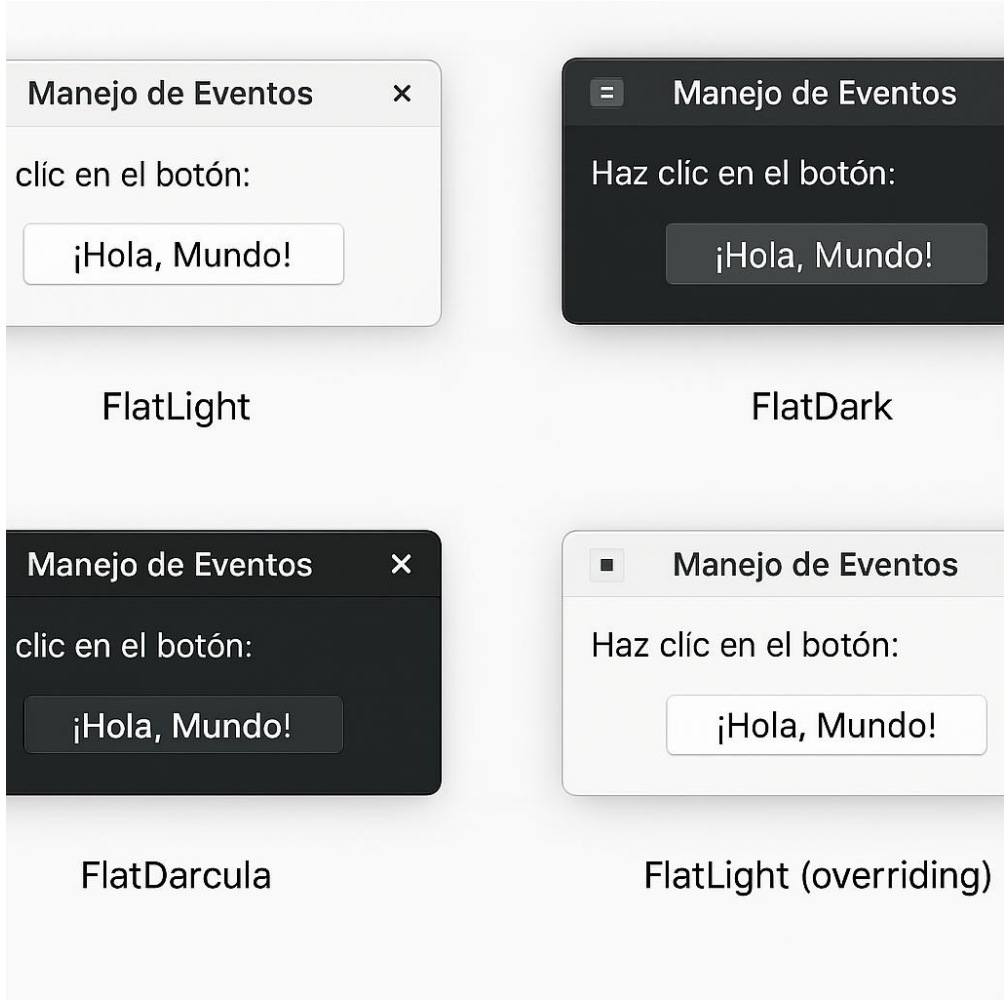


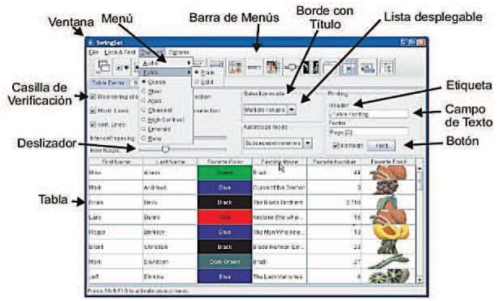
INTERFACES GRÁFICAS DE USUARIO

- ORGANIZACIÓN DE LOS CONTENEDORES.

Apariencia de las ventanas (Look and Feed)

Estilos modernos recomendados: FlatLaf





INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

¿Qué es un contenedor en JavaFX?

En JavaFX, los contenedores son clases que extienden Pane (como VBox, HBox, BorderPane, etc.) y se encargan de organizar los nodos (componentes visuales) dentro de la interfaz.

- En lugar de LayoutManager como en Swing, JavaFX usa directamente clases de disposición que funcionan como paneles ya preconfigurados.

Contenedores más usados en JavaFX

Contenedor	Disposición	Equivalente Swing
HBox	Elementos en fila horizontal	FlowLayout (X_AXIS)
VBox	Elementos en columna vertical	BoxLayout (Y_AXIS)
BorderPane	5 zonas: top, bottom, left, right, center	BorderLayout
GridPane	Rejilla de celdas flexibles	GridBagLayout
StackPane	Apila nodos centrados unos sobre otros	JLayeredPane, CardLayout
FlowPane	Como HBox/VBox, pero se adapta al espacio disponible	FlowLayout
AnchorPane	Posicionamiento absoluto (se fija con coordenadas o anclajes)	null layout (no recomendado)

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Contenedores más usados en JavaFX

- Ejemplo con VBox (organización vertical)

```
VBox layout = new VBox(10); // 10 px de separación vertical
layout.getChildren().addAll(new Label("Nombre:"), new TextField(), new Button("Enviar"));
```

- Ejemplo con BorderPane (zonas)

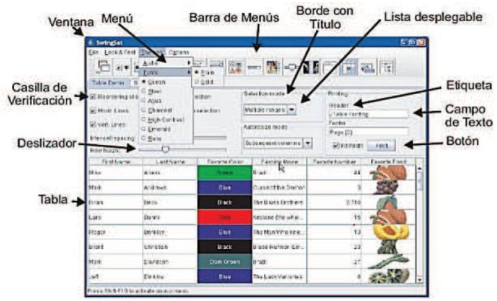
```
BorderPane layout = new BorderPane();
layout.setTop(new Label("Encabezado"));
layout.setCenter(new TextArea());
layout.setBottom(new Button("Aceptar"));
```

- Ejemplo con GridPane (tabla)

```
GridPane grid = new GridPane();
grid.setHgap(10);
grid.setVgap(10);

grid.add(new Label("Usuario:"), 0, 0);
grid.add(new TextField(), 1, 0);
grid.add(new Label("Contraseña:"), 0, 1);
grid.add(new PasswordField(), 1, 1);
```

Cada celda se referencia con (columna, fila)

INTERFACES GRÁFICAS DE USUARIO



ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Contenedores más usados en JavaFX

Diferencias clave respecto a Swing

Aspecto	**Swing**	**JavaFX**
Tipo de contenedor	JPanel con LayoutManager	Pane o subclase (VBox, etc.)
Añadir componente	panel.add(...)	pane.getChildren().add(...)
Espaciado	hgap, vgap o bordes invisibles	parámetros del constructor o setSpacing()
Alineación	controlada por layout + constraints	con setAlignment(...)
Nesting (anidamiento)	Muy habitual con múltiples layouts	Muy natural, más limpio

Buenas prácticas

- ✓ Combina contenedores (VBox + HBox + GridPane) para estructuras complejas
- ✓ Usa Insets para márgenes internos con setPadding(...)
- ✓ Evita AnchorPane salvo para necesidades muy específicas
- ✓ Usa SceneBuilder para diseñar interfaces complejas de forma visual

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Contenedores más usados en JavaFX

```
public class FormularioJavaFX extends Application {
    @Override
    public void start(Stage stage) {
        GridPane grid = new GridPane();
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new Insets(20));

        grid.add(new Label("Nombre:"), 0, 0);
        grid.add(new TextField(), 1, 0);
        grid.add(new Label("Edad:"), 0, 1);
        grid.add(new TextField(), 1, 1);
        grid.add(new Button("Enviar"), 1, 2);

        Scene scene = new Scene(grid, 300, 150);
        stage.setScene(scene);
        stage.setTitle("Formulario JavaFX");
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Apariencia de las ventanas

En Swing...

Se usaba JFrame y sus métodos:

```
JFrame frame = new JFrame("Título");
frame.setSize(400, 300);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
```

En JavaFX...

La clase principal ya hereda de Application, y la ventana se representa con Stage (el escenario).

Componentes clave:

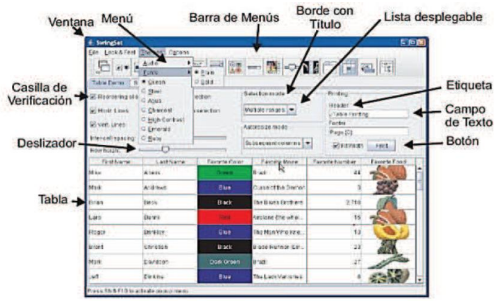
Componente	Descripción
Stage	Ventana principal (como JFrame)
Scene	Contenedor de todos los nodos visuales
Pane o Parent	Raíz del árbol de nodos (layout principal)

INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Apariencia de las ventanas

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MiVentanaJavaFX extends Application {
    @Override
    public void start(Stage stage) {
        Label etiqueta = new Label("Hola Mundo JavaFX");

        StackPane root = new StackPane(etiqueta);
        Scene scene = new Scene(root, 300, 200);

        stage.setTitle("Mi Ventana JavaFX");
        stage.setScene(scene);
        stage.setResizable(false);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```



INTERFACES GRÁFICAS DE USUARIO

ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Apariencia de las ventanas

Métodos del Stage (ventana)

Método JavaFX	Equivalente Swing	Qué hace
stage.setTitle(...)	frame.setTitle(...)	Título de la ventana
stage.setScene(...)	frame.setContentPane(...)	Asigna el contenido
stage.setResizable(...)	frame.setResizable(...)	Permite o no redimensionar
stage.setX(...), setY(...)	frame.setLocation(...)	Posicionamiento manual
stage.centerOnScreen()	frame.setLocationRelativeTo(null)	Centra la ventana
stage.initStyle(...)	— (no hay en Swing directamente)	Permite cambiar el tipo de ventana

¿Qué se puede personalizar?

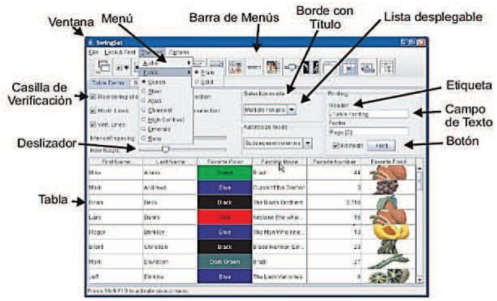
1. Icono de la ventana

```
stage.getIcons().add(new Image("file:icono.png"));
```

2. Estilos CSS

```
scene.getStylesheets().add("estilo.css");
```

INTERFACES GRÁFICAS DE USUARIO



ORGANIZACIÓN DE LOS CONTENEDORES CON JAVAFX.

Apariencia de las ventanas

Estilo visual (CSS en JavaFX)

Puedes crear un archivo estilo.css:

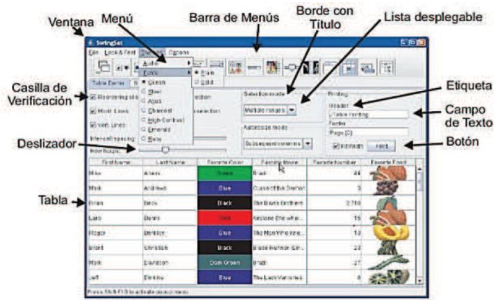
```
.label {
    -fx-font-size: 18px;
    -fx-text-fill: darkblue;
}
.button {
    -fx-background-color: lightgray;
    -fx-border-radius: 5;
}
```

Y aplicarlo con:

```
scene.getStylesheets().add("estilo.css");
```

Elemento	Swing	JavaFX
Estilo global	LookAndFeel (UIManager)	.css aplicado a la Scene
Estilos modernos	FlatLaf, Nimbus, Darcula...	Modificas clases CSS directamente
Estilo por componente	No tan fácil sin UIManager.put	Muy fácil con selectores CSS
Estilo por fichero	.properties o .xml (difícil)	.css puro
Soporte de temas	Limitado, requiere librerías	Nativo, flexible y potente

INTERFACES GRÁFICAS DE USUARIO



CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

¿Qué es un evento?

Un evento es una acción que ocurre en la interfaz gráfica:

- Clic de ratón
- Pulsación de tecla
- Cambio en un campo de texto
- Redimensionar una ventana

Modelo de eventos en Java Swing

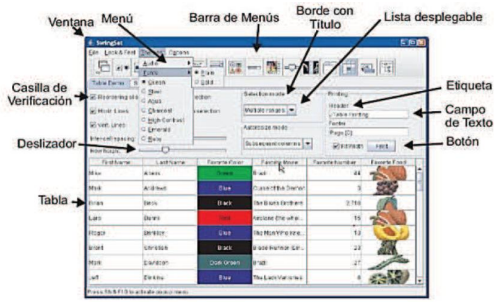
Java Swing usa el modelo Delegado de Eventos (Event Delegation Model):

[Componente GUI]
↓
Genera → [Evento]
↓
Es escuchado por → [Oyente (Listener)]
↓
Dispara → [Método que tú defines]

Ejemplo visual sencillo:

```
Botón "Púlsame" → genera → ActionEvent → detectado por → ActionListener → ejecuta → actionPerformed()
```

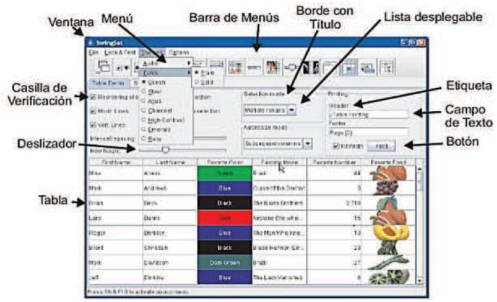
INTERFACES GRÁFICAS DE USUARIO



CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

Los componentes deben estar asociados a determinadas acciones, que pueden ser consecuencia de su pulsación o mostrar los resultados de otra acción.

INTERFACES GRÁFICAS DE USUARIO



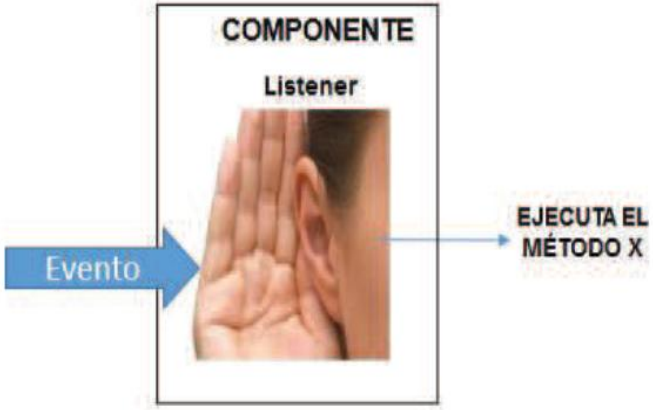
CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

```
JButton boton = new JButton("Púlsame");

boton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("¡Botón pulsado!");
    }
});
```

- El botón genera un evento de tipo ActionEvent.
- El oyente ActionListener escucha ese evento.
- El método actionPerformed() se ejecuta cuando ocurre el evento.

¿Qué elementos intervienen?

Elemento	Qué hace
Componente	El origen del evento (JButton)
Evento	Objeto que describe lo ocurrido
Listener	Interfaz que responde al evento
Método	Código que tú defines



Cómo se asocian eventos?

1. Se implementa una interfaz (ActionListener, MouseListener, etc.)
2. Se registra el listener con add...Listener()
3. Se define el comportamiento en el método correspondiente

INTERFACES GRÁFICAS DE USUARIO

- CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

Los diferentes tipos de Listener tienen efectos específicos para el evento que representan:

Listener	Componentes	Acción a la que responden
ActionListener	1. JButton 2. JTextField 3. JComboBox 4. ...	1. Presionar el botón. 2. Pulsar intro. 3. Elegir una opción. 4. ...
AdjustmentListener	1. JScrollBar 2. ...	Mover la barra de desplazamiento. ...
FocusListener	1. JButton 2. JTextField 3. JComboBox 4. ...	Las acciones de este listener son obtener y perder el foco (colocarnos en el componente e irnos del mismo cuando estaba activo).
ItemListener	1. JCheckbox 2. ...	1. Seleccionar y deseleccionar la opción.
KeyListener	1. JTextField 2. JTextArea 3. ...	Pulsar una tecla cuando el componente tiene el foco.
MouseListener	Múltiples componentes	Acciones como presionar el botón del ratón.
MouseMotionListener	Múltiples componnentes	Acciones como arrastrar (*drag*) o pasar por encima del objeto.
WindowListener	1. JFrame	Acciones relativas a la ventana como por ejemplo cerrarla.

Con lambda (si usas Java 8+)

```
boton.addActionListener(e -> etiqueta.setText("Pulsado"));
```

- ✓ Código más limpio
- ✓ Ideal para acciones simples
- ✓ Aumenta legibilidad si ya se han introducido funciones lambda

INTERFACES GRÁFICAS DE USUARIO

- CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

```
import javax.swing.*;
import java.awt.event.*;

public class EventoBoton {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Evento con botón");
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JButton boton = new JButton("Haz clic");
        JLabel etiqueta = new JLabel("Esperando...");

        // Acción del botón
        boton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                etiqueta.setText("¡Botón pulsado!");
            }
        });

        JPanel panel = new JPanel();
        panel.add(boton);
        panel.add(etiqueta);

        ventana.add(panel);
        ventana.pack();
        ventana.setLocationRelativeTo(null);
        ventana.setVisible(true);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

- CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

Explicación del código paso a paso

Elemento	Qué hace
new JButton("Haz clic")	Crea un botón
addActionListener(...)	Añade el listener
actionPerformed(...)	Método que se ejecuta al pulsar el botón
etiqueta.setText(...)	Cambia el texto del JLabel

Versión alternativa con clase anónima

Ideal si la lógica es simple y solo se usa una vez:

```
boton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        etiqueta.setText("¡Botón pulsado!");
    }
});
```

Versión moderna con lambda (si usas Java 8+)

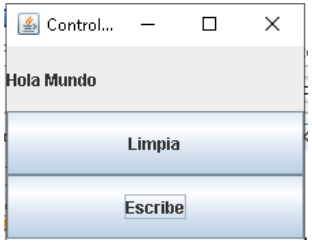
Mucho más legible para alumnos que ya conocen funciones lambda:

```
boton.addActionListener(e -> etiqueta.setText("¡Botón pulsado!"));
```

INTERFACES GRÁFICAS DE USUARIO

- CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

```java
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.UIManager;
import javax.swing.UnsupportedLookAndFeelException;

public class SwingTest {
    private static JLabel label = new JLabel("--");
    private static JButton btnlimpia = new JButton("Limpia");
    private static JButton btnescribe = new JButton("Escribe"
    public static void acciones(ActionEvent e){
        Object obj = e.getSource();
        if(obj == btnlimpia){
            label.setText("");
        }
        if(obj == btnescribe){
            label.setText("Hola Mundo");
        }
    }
```




```java
    public static void main(String [] args) {
        try{
            UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());

        }catch(Exception e){ }

        JFrame frame = new JFrame("Controlando eventos");
        btnlimpia.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e){
                acciones(e);
            }
            });
        btnescribe.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e){
                acciones(e);
            }
            });
        frame.getContentPane().add(label);
        frame.getContentPane().add(btnlimpia);
        frame.getContentPane().add(btnescribe);
        frame.addWindowListener(new WindowAdapter () {
            public void windowsClosing(WindowEvent e){
                System.exit(0);
            }
        });
        frame.setLayout(new GridLayout(0,1));
        frame.pack();
        frame.setVisible(true);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

Vamos a comentar las líneas de código nuevas que han ido apareciendo:

```
try {
UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
} catch (Exception e) { }
```

En las líneas de código anteriores se establece el aspecto general de la aplicación el cual, como antes se indicó, es del tipo multiplataforma.

```
public static void acciones (ActionEvent e) {
       Object obj=e.getSource();
       if (obj ==   btnlimpia) {
        label.setText("");
       }
       if (obj == btnescribe) {
        label.setText("Hola Mundo");
       }
}
```

La función anterior dependiendo del tipo de botón que ha originado el evento escribirá una u otra cosa en la etiqueta de la aplicación. Mediante la función e.getSource() se puede saber el objeto que generó el evento.

INTERFACES GRÁFICAS DE USUARIO

CONCEPTO DE EVENTO Y CONTROLADORES DE EVENTOS

```
btnlimpia.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent e) {
 acciones(e);
} );
btnescribe.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent e) {
 acciones (e);
} );
```

En las líneas anteriores se crea un listener para los dos botones de la aplicación (btnescribe y btnlimpia). Los dos listener ejecutarán el método acciones, el cual distinguirá qué botón ha generado el evento y realizará la acción correspondiente.

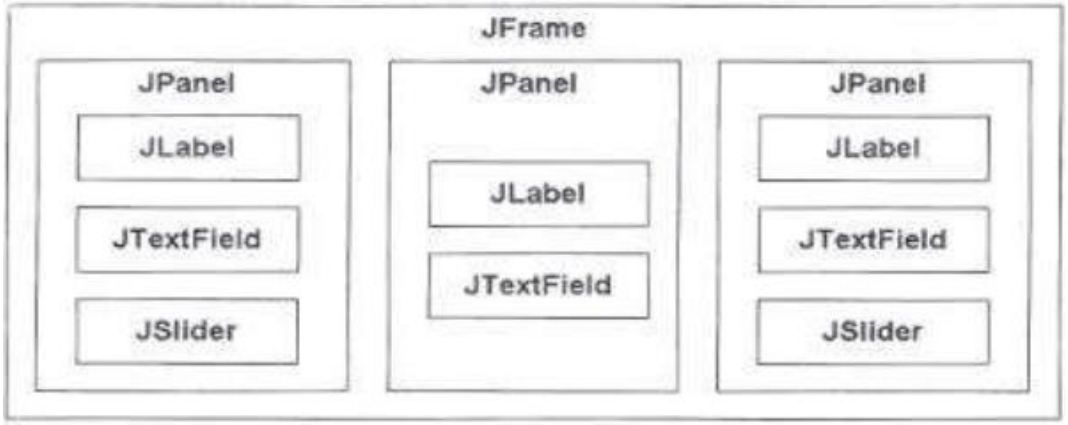
```
frame.addWindowListener(new WindowAdapter() {
public void windowClosing(WindowEvent  e) {
 System.exit(0);
} );
```

Para la ventana el procedimiento es el mismo que para los botones. La acción a ejecutar al cerrar la misma será la terminación de la aplicación con código 0 (todo correcto).

INTERFACES GRÁFICAS DE USUARIO

- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO

Ejm.: Conversión de dólares a euros y viceversa.



INTERFACES GRÁFICAS DE USUARIO

- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO

Ejm.: Conversión de dólares a euros y viceversa.

```java
public class ConversosEurDol {
    static final int min = 0;
    static final int max = 1000;
    static final int init = 0;
    private static JLabel label = new JLabel("1 Euro son en dólares:");
    private static JLabel lbleuros = new JLabel("Euros: ");
    private static JLabel lbldolares = new JLabel("Dolares: ");
    private static JFrame frame = new JFrame();
    private static JPanel panel1 = new JPanel();
    private static JPanel panel2 = new JPanel();
    private static JPanel panel3 = new JPanel();
    private static JTextField txteuro = new JTextField("0");
    private static JTextField txtdolar = new JTextField("0");
    private static JTextField txtcambio = new JTextField("1.36");
    private static JSlider sliderdolar = new JSlider(JSlider.HORIZONTAL, min, max, init
    private static JSlider slidereuro = new JSlider(JSlider.HORIZONTAL, min, max, init)

  public static void cambiotexto(ActionEvent e){
    if(e.getSource() == txteuro){
       float icambio = Float.parseFloat(txteuro.getText());
       icambio = 100*icambio*Float.parseFloat(txtcambio.getText());
       icambio = Math.round(icambio);
       icambio = icambio / 100;
       txtdolar.setText(String.valueOf(icambio));
       sliderdolar.setValue(Math.round(Float.parseFloat(txtdolar.getText())));
       slidereuro.setValue(Math.round(Float.parseFloat(txteuro.getText())));
    }
    if(e.getSource() == txtdolar){
       System.out.println("dentro");
       float icambio = Float.parseFloat(txtdolar.getText());
       icambio = 100*icambio/Float.parseFloat(txtcambio.getText());
       icambio = Math.round(icambio);
       icambio = icambio/100;
       txteuro.setText(String.valueOf(icambio));}}
```

```java
public static void mueveSlider(ChangeEvent e){
   int valor;
   JSlider obj = (JSlider) e.getSource();
   System.out.println(obj.getValueIsAdjusting());
   System.out.println(obj.getValue());
   if(!obj.getValueIsAdjusting()){
      System.out.println(obj.getValue());
      valor = (int) obj.getValue();
      if(obj == sliderdolar){
         txtdolar.setText(String.valueOf(valor));
         float icambio = 100*valor/Float.parseFloat(txtcambio.getText());
                icambio = Math.round(icambio);
                icambio = icambio/100;
                txteuro.setText(String.valueOf(icambio));
                int i = Math.round(icambio);
                slidereuro.setValue(i);
      }
      if(obj == slidereuro){
         txteuro.setText(String.valueOf(valor));
         float icambio = 100*valor*Float.parseFloat(txtcambio.getText());
         icambio = Math.round(icambio);
         icambio = icambio/100;
         txtdolar.setText(String.valueOf(icambio));
         int i = Math.round(icambio);
         slidereuro.setValue(i);
      }
   }
}
```

INTERFACES GRÁFICAS DE USUARIO

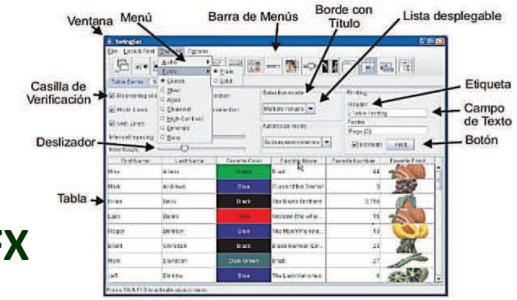
- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO

Ejm.: Conversión de dólares a euros y viceversa.

```
public static void colocaelementos(){
   frame.getContentPane().add(panel1);
   frame.getContentPane().add(panel2);
   frame.getContentPane().add(panel3);
   slidereuro.setBorder(BorderFactory.createTitledBorder("Euro"));
   slidereuro.setMajorTickSpacing(200);
   slidereuro.setMinorTickSpacing(100);
   slidereuro.setPaintTicks(true);
   slidereuro.setPaintLabels(true);
   slidereuro.setEnabled(false);
   sliderdolar.setBorder(BorderFactory.createTitledBorder("Dolares"));
   sliderdolar.setMajorTickSpacing(200);
   sliderdolar.setMinorTickSpacing(100);
   sliderdolar.setPaintTicks(true);
   sliderdolar.setPaintLabels(true);
   sliderdolar.setEnabled(false);
   panel1.add(lbleuros);
   panel1.add(txteuro);
   panel1.add(slidereuro);
   panel2.add(label);
   panel2.add(txtcambio);
   panel3.add(lbldolares);
   panel3.add(txtdolar);
   panel3.add(sliderdolar);
   frame.addWindowListener(new WindowAdapter(){
     public void windowClosing(WindowEvent e){
        System.exit(0);
     } });
   txteuro.addActionListener(new ActionListener(){
      public void actionPerformed(ActionEvent e){
         cambiotexto(e);
      } });
   txtdolar.addActionListener(new ActionListener(){
      public void actionPerformed(ActionEvent e){
         cambiotexto(e);
      } });
```

```
      frame.setLayout(new FlowLayout());
      panel1.setLayout(new GridLayout(0,1));
      panel2.setLayout(new GridLayout(0,1));
      panel3.setLayout(new GridLayout(0,1));
      frame.pack();
      frame.setVisible(true);
   }

   public static void main(String[] args){
      try{
         UIManager.setLookAndFeel(
            UIManager.getCrossPlatformLookAndFeelClassName());
      }catch(Exception e){}
      colocaelementos();
   }
}
```

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

¿Qué es un evento en JavaFX?

Un evento es una acción que ocurre en la interfaz gráfica:

- Pulsar un botón
- Escribir en un campo de texto
- Mover el ratón o pulsar una tecla

JavaFX tiene su propio sistema de eventos, más moderno y limpio que el de Swing.

JavaFX no necesita interfaces como ActionListener.

Se usan métodos como:

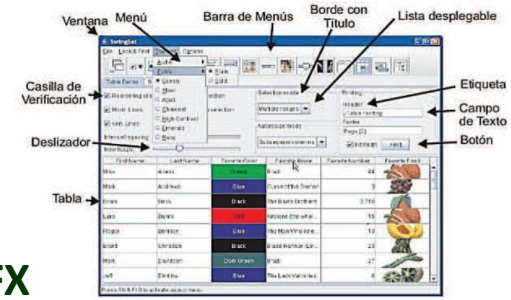
```
boton.setOnAction(evento -> {
    // código que se ejecuta al hacer clic
});
```

Este método registra un manejador de eventos, que se ejecuta cuando ocurre la acción.

JavaFX permite usar **lambdas** para que el código sea más legible y corto, siempre que uses Java 8 o superior.

```
Button boton = new Button("Púlsame");
Label etiqueta = new Label("Esperando...");

// Manejador de eventos con lambda
boton.setOnAction(e -> etiqueta.setText("¡Botón pulsado!"));
```

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

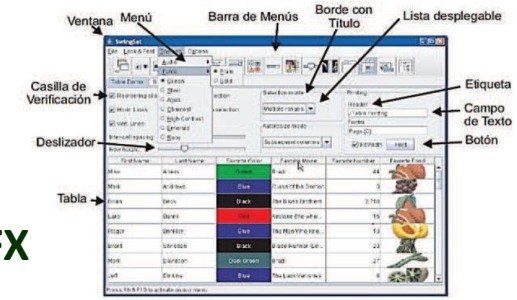
Eventos en JavaFX

Componente	Evento más habitual	Método / Manejador
Button	Clic	setOnAction(e -> {...})
TextField	Pulsar Enter	setOnAction(...)
CheckBox	Cambio de selección	selectedProperty().addListener(...)
Slider	Movimiento del control	valueProperty().addListener(...)
ComboBox	Selección de elemento	setOnAction(...)

Formas de registrar eventos:

- Directamente en código Java con lambdas o métodos.
- Desde SceneBuilder escribiendo #nombreDelMetodo en el panel derecho.
- Desde FXML con onAction="#nombreMetodo" y fx:id para enlazar componentes.

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

Ejemplo básico con botón y etiqueta en JavaFX

Label etiqueta → Muestra un texto inicial

Button boton → Botón que el usuario puede pulsar

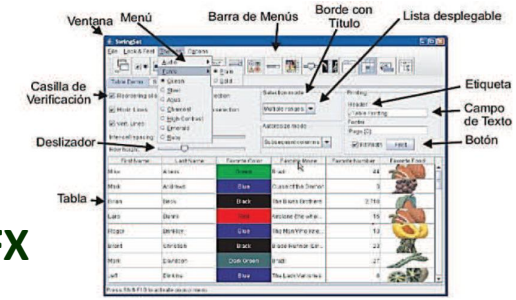
setOnAction(...) → Define qué ocurre cuando se hace clic

VBox(10, ...) → Organiza los elementos verticalmente con 10 px de separación

stage.setScene(...) → Establece la escena en la ventana

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class EventoBasico extends Application {
    @Override
    public void start(Stage stage) {
        Label etiqueta = new Label("Esperando...");
        Button boton = new Button("Haz clic");

        // Manejador de evento con lambda
        boton.setOnAction(e -> etiqueta.setText("¡Botón pulsado!"));

        VBox layout = new VBox(10, boton, etiqueta);
        Scene escena = new Scene(layout, 300, 150);

        stage.setTitle("Evento JavaFX");
        stage.setScene(escena);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

Comparativa de eventos: JavaFX vs Swing

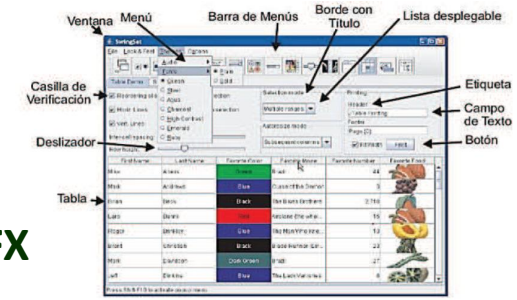
Característica	Swing	JavaFX
Clase del botón	JButton	Button
Evento de clic	ActionEvent	ActionEvent (mismo nombre)
Registro del evento	addActionListener(...)	setOnAction(...)
Código sin lambda	Clase anónima (new ActionListener {...})	Clase anónima o clase externa
Código con lambda	Desde Java 8	Desde Java 8
Estructura general	JFrame, JPanel, JLabel, JButton	Stage, Scene, VBox, Label, Button
Estética moderna	Con FlatLaf (librería externa)	Con CSS y temas integrados

Ventajas de JavaFX en eventos:

- Código más limpio gracias a lambdas
- No se requiere implementar interfaces
- Compatible con Scene Builder (puedes asignar eventos visualmente)

Ambos sistemas permiten separar la lógica de la interfaz, pero JavaFX lo hace más fácilmente gracias a FXML y setOnAction

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

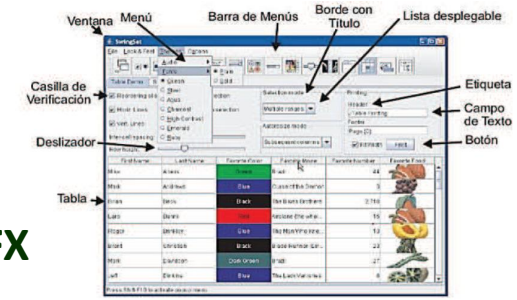
Separación de lógica y vista con FXML en JavaFX

1. Archivo FXML: vista.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<VBox xmlns="http://javafx.com/javafx"
      xmlns:fx="http://javafx.com/fxml"
      fx:controller="ejemplo.Controlador"
      spacing="10" alignment="CENTER" padding="20">

    <Button text="Púlsame" onAction="#alPulsar"/>
    <Label fx:id="etiqueta" text="Esperando..."/>

</VBox>
```

Elemento FXML	Significado
fx:controller	Indica qué clase se encargará del manejo de eventos
onAction="#alPulsar"	Enlaza el clic del botón con el método alPulsar() del controlador
fx:id="etiqueta"	Permite acceder a ese componente desde Java

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

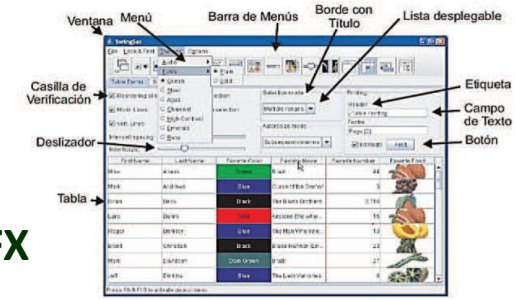
Separación de lógica y vista con FXML en JavaFX

2. Clase controladora: Controlador.java

```
package ejemplo;

import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class Controlador {

    @FXML
    private Label etiqueta;

    @FXML
    private void alPulsar() {
        etiqueta.setText("¡Botón pulsado!");
    }
}
```

Código	Función
@FXML	Permite que JavaFX vincule campos/métodos con el FXML
Label etiqueta	Referencia al componente del FXML
alPulsar()	Se ejecuta cuando se hace clic en el botón

INTERFACES GRÁFICAS DE USUARIO



- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

Separación de lógica y vista con FXML en JavaFX

3. Clase principal: Main.java

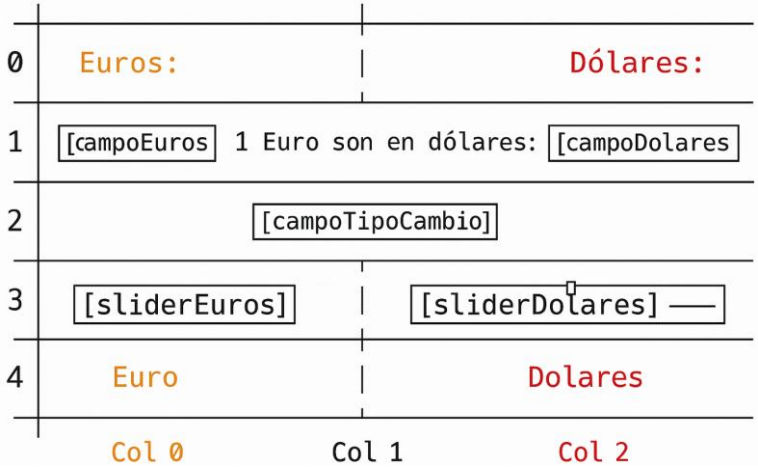
```
package ejemplo;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class Main extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        Parent root = FXMLLoader.load(getClass().getResource("vista.fxml"));
        Scene scene = new Scene(root, 300, 150);
        stage.setTitle("Ejemplo con FXML");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

INTERFACES GRÁFICAS DE USUARIO

- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX

Ejemplo conversión euros-dólares con FXML en JavaFX

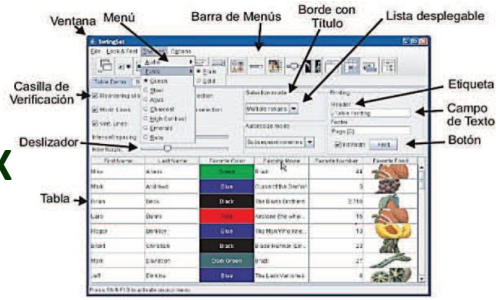
vista.fxml (actualizado y adaptado)




```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.geometry.Insets?>

<GridPane xmlns="http://javafx.com/javafx"
          xmlns:fx="http://javafx.com/fxml"
          fx:controller="ProgramasGUI.ConversorMonetario.Controlador"
          hgap="10" vgap="10" alignment="CENTER">

    <padding>
        <Insets top="20" right="20" bottom="20" left="20"/>
    </padding>

    <!-- Etiquetas -->
    <Label text="Euros:" GridPane.rowIndex="0" GridPane.columnIndex="0"/>
    <Label text="Dólares:" GridPane.rowIndex="0" GridPane.columnIndex="2"/>
    <Label text="1 Euro son en dólares:" GridPane.rowIndex="1" GridPane.columnIndex="1" />

    <!-- Campos de texto -->
    <TextField fx:id="campoEuros" GridPane.rowIndex="1" GridPane.columnIndex="0"/>
    <TextField fx:id="campoTipoCambio" editable="false" GridPane.rowIndex="2" GridPane.columnIndex="1"/>
    <TextField fx:id="campoDolares" GridPane.rowIndex="1" GridPane.columnIndex="2"/>

    <!-- Sliders -->
    <Slider fx:id="sliderEuros" min="0" max="1000" blockIncrement="10"
            showTickMarks="true" showTickLabels="true"
            GridPane.rowIndex="3" GridPane.columnIndex="0"/>
    <Slider fx:id="sliderDolares" min="0" max="1000" blockIncrement="10"
            showTickMarks="true" showTickLabels="true"
            GridPane.rowIndex="3" GridPane.columnIndex="2"/>

    <!-- Etiquetas de sliders -->
    <Label text="Euro" GridPane.rowIndex="4" GridPane.columnIndex="0" />
    <Label text="Dolares" GridPane.rowIndex="4" GridPane.columnIndex="2" />
</GridPane>
```

INTERFACES GRÁFICAS DE USUARIO

- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX



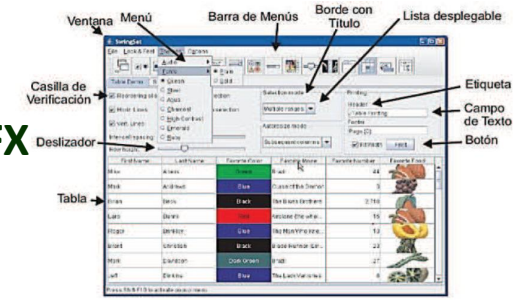
Ejemplo conversión euros-dólares con FXML en JavaFX

Controlador.java

```java
package ProgramasGUI.ConversorMonetario;

import javafx.fxml.FXML;
import javafx.scene.control.Slider;
import javafx.scene.control.TextField;

public class Controlador {
    @FXML private TextField campoEuros;
    @FXML private TextField campoDolares;
    @FXML private TextField campoTipoCambio;
    @FXML private Slider sliderEuros;
    @FXML private Slider sliderDolares;
    // Tasa de conversión fija
    private final double tipoCambio = 1.36;
    // Bandera para evitar actualizaciones recursivas
    private boolean actualizando = false;

    @FXML private void initialize() {
        campoTipoCambio
        .setText(String.format("%.2f", tipoCambio));

        // Listener del slider de euros
        sliderEuros.valueProperty()
        .addListener((obs, oldVal, newVal) -> {
            if (actualizando) return;
            actualizando = true;
            double euros = newVal.doubleValue();
            campoEuros
            .setText(String.format("%.2f", euros));
            double dolares = euros * tipoCambio;
            campoDolares
             .setText(String.format("%.2f", dolares));
            sliderDolares.setValue(dolares);
            actualizando = false;
        });

        // Listener del slider de dólares
        sliderDolares.valueProperty()
         .addListener((obs, oldVal, newVal) -> {
            if (actualizando) return;
            actualizando = true;
            double dolares = newVal.doubleValue();
            campoDolares
             .setText(String.format("%.2f", dolares));
            double euros = dolares / tipoCambio;
            campoEuros
             .setText(String.format("%.2f", euros));
            sliderEuros.setValue(euros);
            actualizando = false;
        });

        // Listener del campo de euros (texto)
        campoEuros.textProperty()
        .addListener((obs, oldVal, newVal) -> {
            if (actualizando) return;
            try {
                actualizando = true;
                double euros = Double.parseDouble(newVal);
                campoDolares.setText(
                    String.format("%.2f", euros * tipoCambio));
                sliderEuros.setValue(euros);
                sliderDolares.setValue(euros * tipoCambio);
            } catch (NumberFormatException e) {
                campoDolares.setText("Valor inválido");
            } finally {
                actualizando = false;
            }
        });
```

INTERFACES GRÁFICAS DE USUARIO

- GENERACIÓN DE PROGRAMAS EN ENTORNO GRÁFICO CON JAVAFX



Ejemplo conversión euros-dólares con FXML en JavaFX

Controlador.java

```
    // Listener del campo de dólares (texto)
    campoDolares.textProperty()
     .addListener((obs, oldVal, newVal) -> {
        if (actualizando) return;
        try {
            actualizando = true;
            double dolares = Double.parseDouble(newVal);
            campoEuros.setText(
                String.format("%.2f", dolares / tipoCambio));
            sliderDolares.setValue(dolares);
            sliderEuros.setValue(dolares / tipoCambio);
        } catch (NumberFormatException e) {
            campoEuros.setText("Valor inválido");
        } finally {
            actualizando = false;
        }
    });
  }
}
```

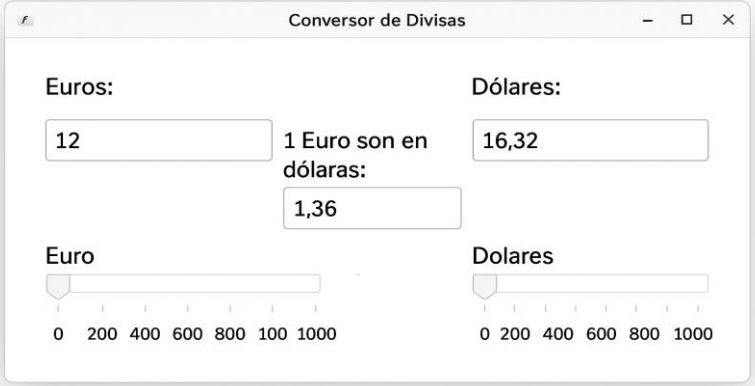
Main.java

```
package ProgramasGUI.ConversorMonetario;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;
public class Main extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        Parent root =
              FXMLLoader.load(getClass().getResource(
      "/ProgramasGUI/ConversorMonetario/vista.fxml"));
        Scene scene = new Scene(root, 550, 300);
        stage.setTitle("Conversor de Divisas");
        stage.setScene(scene);
        stage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

¿Cuándo se usan listeners en JavaFX?

Los listeners son muy útiles cuando:

- Quieres detectar cambios en propiedades (valores, selección, texto...)
- No hay un método setOnAction disponible (como en sliders, checkboxes, campos de texto)
- Quieres observar eventos en tiempo real (sin esperar una acción puntual)



TEMA 7: INTERFACES GRÁFICAS DE USUARIO